Advanced SQL

Q1. What is a Common Table Expression (CTE), and how does it improve SQL query readability

Ans– 1. What is a Common Table Expression (CTE)

A Common Table Expression (CTE) is a temporary result set that you can define inside a SQL query using the `WITH` keyword.
It works like a temporary table that exists only during the execution of that query.

It helps to break complex queries into smaller and easier parts.

Syntax Example:

SQL

```
WITH CTE_Name AS (
  SELECT column1, column2
  FROM table_name
  WHERE condition
)
SELECT *
FROM CTE_Name;
```

2. Example

Suppose we want to find students who scored more than the average score.

Using CTE:

SQL

```
WITH AvgScore AS (
  SELECT AVG(score) AS average_score
  FROM Student_Performance
)

SELECT name, score
FROM Student_Performance
WHERE score > (SELECT average_score FROM
AvgScore);
```

Explanation:

1. `AvgScore` CTE calculates the average score.

2. The main query selects students whose score is greater than the average score.

3. How CTE Improves SQL Query Readability

1. Makes complex queries easier to understand
   Instead of writing long nested subqueries, we divide the query into steps.

2. Improves structure and organization
   Each CTE represents a logical step in the query.

3. Reusable within the same query
   The CTE can be referenced multiple times in the main query.

4. Simplifies debugging
   You can easily check each step of the query.

Definition

A CTE is a temporary named result set created using the `WITH` clause to make complex SQL queries easier to read and manage

Q2. Why are some views updatable while others are read-only? Explain with an example.

Ans–. Updatable View

A view is updatable when we can update, insert, or delete data through the view, and the changes also happen in the original table.

This happens when the view:

- Uses only one table
- Does not use GROUP BY

- Does not use aggregate functions (AVG, SUM, COUNT)

- Does not use JOIN

Example

SQL
```
CREATE VIEW active_cust AS
SELECT *
FROM customer
WHERE active = 1;
```

Now we can update it:

SQL

```
UPDATE active_cust
SET first_name = 'Rahul'
WHERE customer_id = 5;
```

This will also update the customer table.

2. Read-Only View

A view is read-only when we cannot update data through the view.

This happens when the view uses:

- JOIN

- GROUP BY
- Aggregate functions (AVG, SUM, COUNT)
- DISTINCT

Example

SQL
```
CREATE VIEW continent_avg AS
SELECT Continent, AVG(country_pop) AS avg_pop
FROM country
GROUP BY Continent;
```

Here we cannot update:

SQL

```
UPDATE continent_avg
SET avg_pop = 500000;
```

Because it uses AVG() and GROUP BY.

3. Simple Idea

- Simple view (one table) → Updatable
- Complex view (JOIN, GROUP BY, AVG etc.) → Read-only

Some views are updatable because they are created from a single table without complex operations. Views that use JOIN, GROUP BY, or aggregate functions become read-only, so data cannot be updated through them.

Q3. What advantages do stored procedures offer compared to writing raw SQL queries repeatedly?

Ans– What is a Stored Procedure

A Stored Procedure is a pre-written SQL program that is saved in the database and can be executed whenever needed.

Example:

SQL

```
CREATE PROCEDURE Inactive_cust()

BEGIN

SELECT *

FROM customer

WHERE active = 0;

END;
```

To run it:

SQL

CALL Inactive_cust;

Advantages of Stored Procedures

1. Reusability

You can write the query once and use it many times.

Example:
Instead of writing this query again and again:

SQL

SELECT * FROM customer

WHERE active = 0;

You can simply run:

SQL

CALL Inactive_cust;

2. Saves Time

Since the query is already stored in the database, you only need to call the procedure, which saves time and effort.

3. Better Performance

Stored procedures are compiled and optimized by the database, so they often run faster than repeated raw SQL queries.

4. Improved Security

Users can execute the procedure without accessing the actual tables directly, which protects the data.

5. Reduces Code Duplication

Instead of writing the same SQL query in many places, you store it once and reuse it.

Simple Example

Stored Procedure:

SQL

```
CREATE PROCEDURE Daywise_rental()

BEGIN

SELECT

DAYNAME(rental_date) AS dayname_,

COUNT(*) AS total_rentals

FROM rental

GROUP BY dayname_;

END;
```

Run it:

SQL

```
CALL Daywise_rental;
```

Q4.What is the purpose of triggers in a database? Mention one use case where a trigger is essential

Ans– . What is a Trigger

A Trigger is a special type of stored program in a database that automatically executes when a specific event happens on a table.

These events are usually:

- INSERT (when new data is added)
- UPDATE (when data is modified)
- DELETE (when data is removed)

Triggers run automatically, so the user does not need to call them manually.

2. Purpose of Triggers

The main purpose of triggers is to automatically perform actions when data changes in a table.

Triggers help to:

1. Maintain data consistency
2. Automatically update related tables
3. Track changes in data
4. Enforce business rules
5. Create audit logs

3. Example of a Trigger

Suppose we want to record every new customer added to a log table.

Step 1: Create a log table

SQL

```
CREATE TABLE customer_log (
log_id INT AUTO_INCREMENT PRIMARY KEY,
customer_id INT,
action_time DATETIME
);
```

Step 2: Create Trigger

SQL

```
CREATE TRIGGER after_customer_insert
AFTER INSERT ON customer
```

```
FOR EACH ROW
INSERT INTO customer_log(customer_id, action_time)
VALUES (NEW.customer_id, NOW());
```

4. How it Works

When a new customer is inserted into the customer table:

```
INSERT INTO customer VALUES (...);
```

The trigger automatically runs and inserts a record into customer_log.

So the database tracks when a customer was added.

5. Use Case Where Trigger is Essential

Audit logging (tracking data changes).

Example:

- In banking systems
- When money is deposited, withdrawn, or updated
- A trigger can automatically store the transaction history.

This ensures no transaction is missed.

Q5. Explain the need for data modelling and normalization when designing a database

Ans– 1. Data Modelling

Data modelling is the process of organizing and structuring data before creating a database.
It defines how data is stored, how tables are related, and what fields each table will contain.

In simple words, data modelling is like creating a blueprint (plan) of the database.

Need for Data Modelling

1. Organizes Data Properly
   It helps structure data into tables, columns, and relationships.

2. Avoids Confusion
   Clearly defines how tables are connected, making the database easier to understand.

3. Improves Database Design
   Helps developers design an efficient and scalable database.

4. Supports Relationships
   Defines relationships like one-to-one, one-to-many, and many-to-many between tables.

Example

Suppose we are designing a student database.

Instead of storing everything in one table, data modelling divides it into tables like:

- Students
- Courses
- Enrollments

This makes the structure clear and organized.

2. Normalization

Normalization is the process of organizing data in a database to reduce redundancy (duplicate data) and improve data integrity.

It divides large tables into smaller related tables.

Need for Normalization

1. Reduces Data Redundancy
   Prevents storing the same data repeatedly.

2. Improves Data Consistency
   If data is updated once, it is updated everywhere.

3. Saves Storage Space
   Duplicate data is removed.

4. Prevents Data Anomalies
   Avoids problems during insert, update, and delete operations.

3. Conclusion

Data modelling and normalization are important in database design because they organize data efficiently, reduce redundancy, improve data consistency, and make the database easier to manage and maintain.

Dataset (Use for Q6–Q9)

```
CREATE TABLE Products (
 ProductID INT PRIMARY KEY,
 ProductName VARCHAR(100),
```

```
    Category VARCHAR(50),
    Price DECIMAL(10,2)
);

INSERT INTO Products VALUES
(1, 'Keyboard', 'Electronics', 1200),
(2, 'Mouse', 'Electronics', 800),
(3, 'Chair', 'Furniture', 2500),
(4, 'Desk', 'Furniture', 5500);

CREATE TABLE Sales (
    SaleID INT PRIMARY KEY,
    ProductID INT,
    Quantity INT,
    SaleDate DATE,
    FOREIGN KEY (ProductID) REFERENCES
Products(ProductID)
);
```

INSERT INTO Sales VALUES

(1, 1, 4, '2024-01-05'),

(2, 2, 10, '2024-01-06'),

(3, 3, 2, '2024-01-10'),

(4, 4, 1, '2024-01-11');

Q6. Write a CTE to calculate the total revenue for each product (Revenues = Price × Quantity), and return only products where revenue > 3000

Ans– Step-by-Step Explanation

Step 1: Create a View

SQL

```
WITH RevenueCTE AS (
  SELECT
     p.ProductID,
     p.ProductName,
     p.Price,
     s.Quantity,
     (p.Price * s.Quantity) AS Revenue
  FROM Products p
  JOIN Sales s
```

```
  ON p.ProductID = s.ProductID
)

SELECT ProductID, ProductName, Revenue
FROM RevenueCTE
WHERE Revenue > 3000;
```

Easy Explanation Step-by-Step

1 What is CTE

CTE (Common Table Expression) is a temporary result table created using `WITH`.
It helps make complex queries easier to read and manage.

WITH RevenueCTE AS (...)

Here RevenueCTE is the temporary table name.

2 Joining the Tables

```
FROM Products p
JOIN Sales s
ON p.ProductID = s.ProductID
```

We join Products and Sales tables because:

- Products table → contains `Price`
- Sales table → contains `Quantity`

We need Price × Quantity to calculate revenue.

3 Calculating Revenue

(p.Price * s.Quantity) AS Revenue

Formula given in question:

Revenue = Price × Quantity

Example using your data:

Product	Price	Quantity	Revenue
Keyboard	1200	4	4800
Mouse	800	10	8000
Chair	2500	2	5000
Desk	5500	1	5500

4 Filtering Revenue > 3000

WHERE Revenue > 3000

This condition returns only products whose revenue is greater than 3000.

So the output will be:

ProductID	ProductName	Revenue
1	Keyboard	4800
2	Mouse	8000
3	Chair	5000
4	Desk	5500

(All are greater than 3000)

Final Logic

1. Create CTE `RevenueCTE`

2. Join `Products` and `Sales`

3. Calculate `Revenue = Price × Quantity`

4. Select rows where `Revenue > 3000`

Q7.Create a view named VW_CategorySummary that shows: Category, TotalProducts, AveragePrice.

Ans– 1.Step-by-Step Explanation

Step 1: Create a View

SQL

CREATE VIEW VW_CategorySummary AS

- `CREATE VIEW` is used to create a virtual table.

- `VW_CategorySummary` is the name of the view.

- A view stores a query, not the actual data.

Step 2: Select Required Columns

SQL
SELECT
Category,
COUNT(ProductID) AS TotalProducts,
AVG(Price) AS AveragePrice

Here we calculate:

- COUNT(ProductID) → total number of products
- AVG(Price) → average price of products

Step 3: Use the Table

SQL
FROM Products

All product details like Category and Price are stored in the Products table.

Step 4: Group the Data

SQL
GROUP BY Category;

GROUP BY groups products based on Category.

So calculation happens for each category separately.

2 Data from the Table

ProductName	Category	Price

Keyboard	Electronics	1200
Mouse	Electronics	800
Chair	Furniture	2500
Desk	Furniture	5500

3 Final Output of the View

Category	TotalProducts	AveragePrice
Electronics	2	1000
Furniture	2	4000

Explanation:

Electronics

- Products: Keyboard, Mouse
- TotalProducts = 2
- AveragePrice = (1200 + 800) / 2 = 1000

Furniture

- Products: Chair, Desk
- TotalProducts = 2
- AveragePrice = (2500 + 5500) / 2 = 4000

Q8. Create an updatable view containing ProductID, ProductName, and Price.Then update the price of ProductID = 1 using the view

Ans— 1.Create an Updatable View

First, create a view that shows ProductID, ProductName, and Price from the `Products` table.

SQL

CREATE VIEW Product_View AS

SELECT ProductID, ProductName, Price

FROM Products;

Explanation:

- `CREATE VIEW Product_View` → Creates a virtual table (view) called Product_View.
- `SELECT ProductID, ProductName, Price` → The view will display only these three columns.
- `FROM Products` → Data is taken from the Products table.
- This view is updatable because it uses a single table and no aggregate functions.

2 Update the Price Using the View

SQL

UPDATE Product_View

SET Price = 1500

WHERE ProductID = 1;

Explanation:

- `UPDATE Product_View` → We update the data through the view instead of the original table.
- `SET Price = 1500` → Changes the price to 1500.
- `WHERE ProductID = 1` → Only the product with ProductID 1 (Keyboard) will be updated.

3 Result

After executing the update, the Products table will also be updated automatically because the view is linked to it.

ProductID	ProductName	Category	Price
1	Keyboard	Electronics	1500
2	Mouse	Electronics	800
3	Chair	Furniture	2500
4	Desk	Furniture	5500

Q9. Create a stored procedure that accepts a category name and returns all products belonging to that category.

Ans– SQL Query

SQL

```
DELIMITER //

CREATE PROCEDURE GetProductsByCategory(IN
cat_name VARCHAR(50))
BEGIN
SELECT ProductID, ProductName, Category, Price
FROM Products
WHERE Category = cat_name;
END //

DELIMITER ;
```

Explanation

1. DELIMITER //

- Changes the symbol used to end the SQL command.
- This is needed when writing a stored procedure.

2. CREATE PROCEDURE GetProductsByCategory

- Creates a stored procedure named GetProductsByCategory.

3. IN cat_name VARCHAR(50)

- `IN` means input parameter.
- `cat_name` is the category name given by the user.

4. SELECT statement

- Retrieves ProductID, ProductName, Category, and Price from the Products table.

5. WHERE Category = cat_name

- Shows only the products that belong to the given category.

6. END

- Marks the end of the stored procedure.

How to Run the Procedure

SQL

```
CALL GetProductsByCategory('Electronics');
```

Result Example

ProductID	ProductName	Category	Price
1	Keyboard	Electronics	1200
2	Mouse	Electronics	800

Q10.Create an After Delete trigger on the Product table that archives deleted product rows into a new table Product Archive.The archive should store ProductID, ProductName, Category, Price, and DeletedAt timestamp.

Ans– 1. Create Archive Table

First create a table where deleted product records will be stored.

SQL

```
CREATE TABLE ProductArchive (
  ProductID INT,
  ProductName VARCHAR(100),
  Category VARCHAR(50),
  Price DECIMAL(10,2),
  DeletedAt TIMESTAMP
);
```

Explanation

This table will store deleted product information.

Columns:

- ProductID → ID of the deleted product
- ProductName → Name of the product
- Category → Product category
- Price → Price of the product
- DeletedAt → Time when the product was deleted

2. Create AFTER DELETE Trigger

SQL

```
DELIMITER //

CREATE TRIGGER AfterProductDelete
AFTER DELETE ON Products
FOR EACH ROW
BEGIN
INSERT INTO ProductArchive
(ProductID, ProductName, Category, Price, DeletedAt)
VALUES
(OLD.ProductID, OLD.ProductName, OLD.Category,
OLD.Price, NOW());
END //

DELIMITER ;
```

3.Explanation

1 CREATE TRIGGER AfterProductDelete

- Creates a trigger named AfterProductDelete.

2 AFTER DELETE ON Products

- The trigger runs after a row is deleted from the `Products` table.

3 FOR EACH ROW

- The trigger works for every deleted row.

4 OLD Keyword

- `OLD` refers to the data that existed before deletion.

Example:

- `OLD.ProductID`
- `OLD.ProductName`
- `OLD.Category`
- `OLD.Price`

These values are copied to the archive table.

5 NOW()

- `NOW()` stores the current date and time when the product is deleted.

4. How It Works (Example)

If we run:

SQL

```
DELETE FROM Products
WHERE ProductID = 2;
```

The product Mouse will be deleted from `Products`.

But the trigger will automatically store it in ProductArchive.

5. ProductArchive Table Result

ProductID	ProductName	Category	Price	DeletedAt
2	Mouse	Electronics	800	2024-01-20